

Deadswitch — Product Requirements Document (PRD)

Version: 1.0.0 **Date:** April 14, 2026 **Author:** Deadswitch Labs **Status:** Draft **Last Updated:** April 14, 2026

Table of Contents

1. [Executive Summary](#)
 2. [Product Vision & Goals](#)
 3. [Target Users & Personas](#)
 4. [Scope & Constraints](#)
 5. [Feature Requirements](#)
 6. [User Stories & Acceptance Criteria](#)
 7. [Information Architecture](#)
 8. [Page-by-Page Specifications](#)
 9. [Smart Contract Specifications](#)
 10. [Backend Specifications](#)
 11. [API Specifications](#)
 12. [Database Schema](#)
 13. [Notification System](#)
 14. [Crank Bot Specifications](#)
 15. [Non-Functional Requirements](#)
 16. [Tech Stack & Architecture Decisions](#)
 17. [Environment & Configuration](#)
 18. [Third-Party Integrations](#)
 19. [Error Handling & Edge Cases](#)
 20. [Testing Strategy](#)
 21. [Deployment Strategy](#)
 22. [Analytics & Metrics](#)
 23. [Development Phases & Milestones](#)
 24. [Risk Register](#)
 25. [Open Questions & Decisions](#)
 26. [Appendix](#)
-

1. Executive Summary

1.1 What We're Building

Deadswitch is a non-custodial onchain inheritance protocol on Solana. Users deposit crypto assets into a smart contract vault with instructions for how those assets should be distributed to named beneficiaries if the user's wallet becomes inactive for a configurable period.

1.2 Why We're Building It

- ~\$140B+ in crypto is permanently lost, much of it from deceased or incapacitated holders.
- No inheritance tool exists on Solana (zero competition).
- Existing solutions (Sarcophagus on Ethereum) reveal secrets rather than transferring assets, require manual check-ins, and suffer from low adoption due to cold-start node network problems.
- This is our submission for the Solana Frontier Hackathon (deadline: May 11, 2026).

1.3 Success Criteria for Hackathon

Criteria	Target
Working devnet deployment	Full create → monitor → redistribute flow
Demo video	3-5 minutes, showing end-to-end flow
Open-source repository	Public GitHub, MIT license, clean README
Whitepaper	Published (completed)
Code quality	TypeScript strict, Anchor tests passing, no critical bugs
UX	Non-technical person can create a vault in under 5 minutes

1.4 Key Deadlines

Milestone	Date
PRD finalized	April 14, 2026
Anchor program complete + tested	April 24, 2026
Backend + Helius integration complete	April 30, 2026
Frontend complete	May 5, 2026
Hackathon registration deadline	May 4, 2026
Polish, demo video, documentation	May 8, 2026
Hackathon submission deadline	May 11, 2026
Winners announced	June 23, 2026

2. Product Vision & Goals

2.1 Vision Statement

"Your crypto shouldn't die with you."

Deadswitch makes crypto inheritance as simple as setting up a beneficiary on a bank account — but without trusting a bank, a company, or a third party. Set it once, forget it, and know that your loved ones are protected.

2.2 Product Goals

#	Goal	Metric
G1	Make crypto inheritance accessible to non-technical users	Vault creation completes in <5 minutes, <5 clicks after wallet connect
G2	Ensure beneficiaries need zero crypto knowledge to receive assets	Beneficiary receives tokens without any protocol interaction
G3	Maintain non-custodial, trustless operation	No admin keys can access user funds; all critical logic is onchain

G4	Demonstrate viability as a business	Clear freemium model with defined pricing tiers
G5	Win or place in Solana Frontier Hackathon	Score high on Functionality, Novelty, UX, Business Plan, Impact, Open-source

2.3 Design Principles

1. **Safety over speed.** Every design decision should favor preventing false triggers (premature redistribution) over fast execution. A delayed inheritance is inconvenient; a premature one is catastrophic.
 2. **Zero burden on beneficiaries.** The beneficiary experience is: tokens appear in their wallet. Nothing more. No app downloads, no claim pages, no seed phrases.
 3. **Passive proof-of-life.** The owner should never need to "check in." Normal wallet usage IS the check-in.
 4. **Transparent and auditable.** All vault state is onchain. Anyone can verify the protocol's behavior. Open-source everything.
 5. **Progressive disclosure.** Simple defaults for new users (90 days, 7-day grace, 0.1% crank fee). Advanced options available but not required.
-

3. Target Users & Personas

3.1 Persona 1: The HODLer (Primary)

Attribute	Detail
Name	Alex
Age	28-45
Crypto experience	Intermediate — holds SOL, USDC, some DeFi positions
Portfolio size	\$5,000 – \$100,000
Pain point	"If something happens to me, my family has no idea how to access my crypto"
Technical comfort	Can use Phantom wallet, swap on Jupiter, but not a developer
What they want	Set-and-forget inheritance plan that costs less than a lawyer
Behavior	Uses wallet 2-5 times per month for swaps, transfers, staking

3.2 Persona 2: The DeFi Power User (Secondary)

Attribute	Detail
Name	Sam
Age	22-35
Crypto experience	Advanced — multiple wallets, DeFi positions, NFTs
Portfolio size	\$50,000 – \$500,000+
Pain point	"I have assets across multiple wallets and protocols. If I get hit by a bus, millions in crypto vanish"

Technical comfort	Developer or power user, uses CLI, understands PDAs
What they want	Multiple vaults with different configurations, API access, detailed control
Behavior	Daily wallet activity, manages multiple positions

3.3 Persona 3: The Beneficiary (Passive)

Attribute	Detail
Name	Maya (Alex's wife)
Age	25-70
Crypto experience	Minimal to none
Pain point	"My husband mentioned something about crypto but I wouldn't know how to access it"
Technical comfort	Has a smartphone, can install an app if guided
What they want	To receive her inheritance without needing to understand blockchain
Interaction with Deadswitch	None. She receives tokens in a wallet that Alex set up for her (or that she creates with guidance)

3.4 Persona 4: The Crank Operator (Infrastructure)

Attribute	Detail
Name	Bot Runner
Age	N/A (automated)
Crypto experience	Advanced — runs MEV bots, liquidation bots
Pain point	"I need new sources of bot income on Solana"
What they want	Profitable, low-risk crank execution opportunities
Interaction with Deadswitch	Runs automated bot that monitors triggered vaults and submits execute transactions

4. Scope & Constraints

4.1 In Scope (Hackathon MVP)

Category	Features
Smart Contract	create_vault, update_vault, top_up_vault, cancel_vault, record_heartbeat, execute_redistribution
Assets	Native SOL, USDC (SPL), USDT (SPL)
Frontend	Landing page, wallet connect, vault creation wizard, dashboard, vault detail page
Backend	Helius webhook receiver, heartbeat submitter, vault indexer, basic crank bot

Notifications	Email alerts (50%, 75%, 90% inactivity thresholds)
Network	Solana Devnet (with mainnet-ready architecture)

4.2 Out of Scope (Post-Hackathon)

Feature	Reason
Arbitrary SPL token support (beyond SOL/USDC/USDT)	MVP scope — add after hackathon
NFT / cNFT support	Complex asset handling — v2
Staked SOL positions	Requires unstaking flow — v2
LP token positions	Requires withdrawal flow — v2
Multi-chain (EVM, Bitcoin)	Post-hackathon expansion
Premium subscription billing	No payment needed for MVP
B2B / institutional features	Post-hackathon
Telegram bot notifications	Email-only for MVP
Mobile native app	Responsive web is sufficient for MVP
Legal compliance features	Post-hackathon, jurisdiction-specific
Multi-language / i18n	English only per hackathon rules

4.3 Constraints

Constraint	Detail
Timeline	~27 days (April 14 → May 11, 2026)
Team size	1 developer (Wira)
Budget	Near zero — use free tiers (Helius free plan, Vercel free, etc.)
Network	Devnet for hackathon demo; mainnet-ready architecture
Language	All content must be in English (hackathon requirement)
Open source	Required for hackathon scoring (open-source criteria)
Non-custodial	Core principle — protocol must never hold private keys or have admin drain capability

5. Feature Requirements

5.1 Feature Map

Features are organized by priority using MoSCoW method:

Must Have (M) — Hackathon MVP

ID	Feature	Description
F-001	Wallet Connection	Connect Solana wallet (Phantom, Backpack, Solflare) via Wallet Adapter
F-002	Vault Creation	Create a vault with beneficiaries, asset config, inactivity window, grace period
F-003	Asset Deposit	Deposit SOL and/or USDC/USDT into the vault during creation
F-004	Vault Dashboard	View all vaults owned by connected wallet with status, TVL, time remaining
F-005	Vault Detail View	View full vault details: beneficiaries, assets, activity log, status timeline
F-006	Vault Cancellation	Cancel an active vault and reclaim all deposited assets
F-007	Vault Update	Modify beneficiaries, percentages, inactivity window, grace period
F-008	Vault Top-Up	Add more assets to an existing vault
F-009	Heartbeat Recording	Backend records heartbeats when Helius webhook detects owner wallet activity
F-010	Redistribution Execution	Crank bot (or anyone) can execute redistribution when conditions are met
F-011	Landing Page	Marketing page explaining what Deadswitch is, how it works, and CTA to launch app
F-012	Devnet Faucet Integration	Easy access to devnet SOL/USDC for testing

Should Have (S) — Enhances hackathon score

ID	Feature	Description
F-013	Email Alerts	Send email notifications at 50%, 75%, 90% inactivity thresholds
F-014	Manual Heartbeat	Owner can manually submit a heartbeat via the frontend (button click)
F-015	Vault Activity Log	Show history of all heartbeats with timestamps and tx signatures
F-016	Responsive Mobile UI	Full mobile browser support for vault creation and dashboard
F-017	Transaction History	Show all vault-related transactions (creation, heartbeats, updates, execution)
F-018	Toast Notifications	In-app feedback for all user actions (success, error, pending)

Could Have (C) — Nice to have if time permits

ID	Feature	Description
F-019	Telegram Alerts	Telegram bot for push notifications
F-020	Vault Sharing	Shareable link to view vault status (read-only, no sensitive data)
F-021	Beneficiary Notification	Email notification to beneficiaries after redistribution
F-022	Dark/Light Theme	Theme toggle for UI
F-023	Vault Templates	Pre-built configurations (e.g., "50/50 split", "Family plan")
F-024	Protocol Stats Page	Public dashboard showing total vaults, TVL, redistributions completed

Won't Have (W) — Explicitly excluded from MVP

ID	Feature	Description
F-025	Premium billing	No payment processing in MVP
F-026	Arbitrary SPL tokens	Only SOL, USDC, USDT in MVP
F-027	NFT vaults	Not in MVP
F-028	Multi-chain	Solana only
F-029	API keys / developer API	Not in MVP
F-030	Admin panel	Not in MVP

6. User Stories & Acceptance Criteria

6.1 Wallet Connection

US-001: Connect Wallet

As a vault owner, **I want to** connect my Solana wallet to Deadswitch, **So that** I can create and manage inheritance vaults.

Acceptance Criteria:

- ☐ User can connect via Phantom, Backpack, or Solflare wallet.
- ☐ Connected wallet address is displayed in the header (truncated: `7xKp...3nFq`).
- ☐ User can disconnect wallet from the header menu.
- ☐ If no wallet extension is detected, show a helpful message with download links.
- ☐ Wallet connection persists across page navigation within the same session.
- ☐ Disconnecting clears all session state.

US-002: Wallet Auto-Detection

As a user, **I want** the app to detect my installed wallets automatically, **So that** I can connect with one click.

Acceptance Criteria:

- ☐ Wallet Adapter detects all installed Solana wallets.
 - ☐ If only one wallet is installed, it is pre-selected.
 - ☐ If multiple wallets are installed, show a selection modal.
 - ☐ "Connect Wallet" button is prominently placed in the header and on the landing page CTA.
-

6.2 Vault Creation

US-003: Create a Basic Vault

As a vault owner, **I want to** create an inheritance vault with beneficiaries, assets, and timing, **So that** my crypto is automatically distributed if I become inactive.

Acceptance Criteria:

- ☐ Vault creation follows a step-by-step wizard (Steps: Name → Beneficiaries → Assets → Timing → Review → Confirm).
- ☐ Each step validates input before allowing progression to the next step.
- ☐ User can navigate back to previous steps without losing data.

- ☐ Final step shows a complete summary before signing.
- ☐ Single transaction signature creates the vault and deposits assets.
- ☐ After successful creation, user is redirected to the vault detail page.
- ☐ Toast notification confirms successful vault creation.

US-004: Name the Vault

As a vault owner, **I want to** give my vault a human-readable name and optional note, **So that** I can identify it and leave a message for my beneficiaries.

Acceptance Criteria:

- ☐ Name field: required, 1-64 characters, plaintext only.
- ☐ Note field: optional, 0-256 characters, plaintext only.
- ☐ Character counter shown for both fields.
- ☐ Placeholder text suggests examples (e.g., "Family Inheritance", "Emergency Fund").
- ☐ Note placeholder: "Optional message to your beneficiaries."

US-005: Add Beneficiaries

As a vault owner, **I want to** add one or more beneficiaries with wallet addresses and percentage shares, **So that** my assets are split correctly during redistribution.

Acceptance Criteria:

- ☐ Minimum 1 beneficiary, maximum 10 beneficiaries.
- ☐ Each beneficiary requires: wallet address (Solana public key), display name (1-32 chars), share percentage.
- ☐ Wallet address validation: must be a valid base58-encoded Solana public key (32 bytes).
- ☐ Wallet address must not be the vault owner's own address (show error: "You cannot be your own beneficiary").
- ☐ Wallet address must not be the zero address.
- ☐ Duplicate wallet addresses are not allowed.
- ☐ Share percentages must be whole numbers between 1 and 99.
- ☐ Total shares must equal exactly 100%. Show running total with visual indicator (green = 100%, red = not 100%).
- ☐ "Add Beneficiary" button adds a new row.
- ☐ "Remove" button on each row (except when only 1 beneficiary remains).
- ☐ Visual pie chart or bar showing the percentage split updates in real-time.
- ☐ Paste wallet address from clipboard support.

US-006: Select Assets to Deposit

As a vault owner, **I want to** choose which tokens and how much to deposit into the vault, **So that** the right assets are distributed to my beneficiaries.

Acceptance Criteria:

- ☐ Show all supported tokens in the connected wallet with current balances.
- ☐ Supported tokens for MVP: SOL, USDC, USDT.
- ☐ Each token row shows: token icon, token symbol, wallet balance, input field for amount.
- ☐ "Max" button fills the input with the full wallet balance (minus rent-exempt minimum for SOL: reserve 0.05 SOL for gas/rent).
- ☐ At least one token must have a non-zero deposit amount.
- ☐ Amount must be > 0 and $\leq$ wallet balance.
- ☐ Show estimated USD value for each token (using onchain price or CoinGecko API).

- ☐ Show total estimated USD value of the vault at the bottom.
- ☐ Validate that the user has enough SOL remaining for transaction fees (~0.01 SOL buffer).
- ☐ If user has zero balance for a token, the row is shown but disabled with "0 balance" label.

US-007: Configure Timing

As a vault owner, **I want to** set my inactivity window and grace period, **So that** I control how long the system waits before redistributing.

Acceptance Criteria:

- ☐ Inactivity window: dropdown or slider with options: 30, 60, 90, 120, 180, 365 days.
- ☐ Default: 90 days (pre-selected, highlighted as "Recommended").
- ☐ Each option shows a brief explanation:
 - 30 days: "For daily active traders"
 - 60 days: "For weekly active users"
 - 90 days: "Recommended for most users"
 - 120 days: "For monthly active users"
 - 180 days: "For long-term holders"
 - 365 days: "Maximum caution"
- ☐ Grace period: dropdown with options: 1, 3, 7, 14, 30 days. Default: 7 days.
- ☐ Explanation text: "After your inactivity window passes, you get an additional grace period before redistribution can happen."
- ☐ Crank fee: shown as advanced option (collapsed by default). Slider: 0.01% - 5%. Default: 0.1%.
- ☐ Explanation text for crank fee: "A small fee paid to the person who submits the redistribution transaction. Higher fees mean faster execution."
- ☐ Show total timeline summary: "If your wallet is inactive for [X] days + [Y] day grace period = [total] days before redistribution."

US-008: Review and Confirm Vault

As a vault owner, **I want to** review all vault settings before signing the transaction, **So that** I can verify everything is correct.

Acceptance Criteria:

- ☐ Summary shows: vault name, note, all beneficiaries (name, address, percentage), all assets (token, amount, USD value), inactivity window, grace period, crank fee, estimated gas cost.
- ☐ Each section has an "Edit" button that returns to the relevant step.
- ☐ "Create Vault" button is prominent (primary action).
- ☐ Button text changes to "Creating..." with spinner during transaction.
- ☐ If transaction fails, show error message with reason and "Try Again" button.
- ☐ If transaction succeeds, show success state and redirect to vault detail page after 2 seconds.
- ☐ Confirmation modal before signing: "You are about to deposit [X] SOL and [Y] USDC into this vault. These assets will be locked until you cancel the vault or redistribution occurs. Continue?"

6.3 Vault Management

US-009: View Vault Dashboard

As a vault owner, **I want to** see all my vaults in a dashboard view, **So that** I can monitor their status at a glance.

Acceptance Criteria:

- ☐ Show a card or table for each vault owned by the connected wallet.
- ☐ Each vault entry shows: vault name, status badge (Active/Warning/Triggered/Executed/Cancelled), total value locked (USD estimate), time since last activity, time remaining before trigger, number of beneficiaries.
- ☐ Status badge colors: Active = green, Warning = yellow/amber, Triggered = red, Executed = gray, Cancelled = gray.
- ☐ Vaults sorted by status priority: Warning > Active > Triggered > Executed > Cancelled.
- ☐ "Create New Vault" button prominently visible.
- ☐ If no vaults exist, show empty state: "You haven't created any vaults yet. Create your first vault to protect your crypto."
- ☐ Clicking a vault card navigates to the vault detail page.

US-010: View Vault Detail

As a vault owner, **I want to** see full details of a specific vault, **So that** I can monitor its configuration and activity.

Acceptance Criteria:

- ☐ Show vault name, status, creation date.
- ☐ Status section: current status badge, visual timeline bar showing progress through inactivity window (e.g., "Day 23 of 90"), countdown timer showing days/hours remaining.
- ☐ Beneficiaries section: table with name, wallet address (truncated + copy button), share percentage, estimated value they'd receive at current vault balance.
- ☐ Assets section: list of deposited tokens with amounts and USD estimates.
- ☐ Activity log section: chronological list of heartbeat events with timestamps and Solana Explorer links to the source transactions.
- ☐ Configuration section: inactivity window, grace period, crank fee.
- ☐ Action buttons: "Update Vault", "Top Up", "Cancel Vault" (with confirmation), "Manual Heartbeat."
- ☐ If vault is Executed: show redistribution details (who received what, tx signature, execution timestamp).
- ☐ If vault is Cancelled: show cancellation timestamp and return tx signature.

US-011: Update Vault Configuration

As a vault owner, **I want to** modify my vault's beneficiaries, percentages, and timing, **So that** I can keep my inheritance plan current.

Acceptance Criteria:

- ☐ Only available for Active or Warning vaults.
- ☐ Opens an edit form pre-filled with current vault configuration.
- ☐ Can modify: vault name, note, beneficiaries (add/remove/change), share percentages, inactivity window, grace period, crank fee.
- ☐ Cannot modify for MVP: deposited assets (use top-up or cancel+recreate instead).
- ☐ Same validation rules as vault creation (US-004 through US-007).
- ☐ Updating resets the inactivity timer (since it requires owner signature = proof of life).
- ☐ Show confirmation: "Updating your vault will also reset your inactivity timer. Continue?"
- ☐ Single transaction signature to apply changes.

US-012: Top Up Vault

As a vault owner, **I want to** add more assets to an existing vault, **So that** I can increase the amount my beneficiaries will receive.

Acceptance Criteria:

- ☐ Only available for Active or Warning vaults.
- ☐ Shows current vault assets and wallet balances.
- ☐ User selects token(s) and amount(s) to add.

- ☐ Same validation as US-006 (amount > 0, ≤ balance, SOL buffer for gas).
- ☐ Topping up resets the inactivity timer.
- ☐ Single transaction signature to deposit additional assets.
- ☐ Vault detail page updates to reflect new balances.

US-013: Cancel Vault

As a vault owner, **I want to** cancel a vault and reclaim all deposited assets, **So that** I can recover my funds if I no longer want the vault.

Acceptance Criteria:

- ☐ Only available for Active or Warning vaults.
- ☐ Confirmation dialog: "Are you sure you want to cancel this vault? All deposited assets ([X] SOL, [Y] USDC) will be returned to your wallet. This action cannot be undone."
- ☐ Requires typing vault name to confirm (prevent accidental cancellation).
- ☐ Single transaction: transfers all vault assets back to owner, closes vault ATAs (rent returned), sets vault status to Cancelled.
- ☐ Toast notification: "Vault cancelled. All assets returned to your wallet."
- ☐ Vault appears in dashboard with "Cancelled" status badge (greyed out).

US-014: Manual Heartbeat

As a vault owner, **I want to** manually send a heartbeat, **So that** I can reset my inactivity timer without making a separate wallet transaction.

Acceptance Criteria:

- ☐ "I'm Still Here" button on vault detail page.
- ☐ Submits a `record_heartbeat` transaction signed by the owner.
- ☐ Timer resets to zero immediately.
- ☐ Toast notification: "Heartbeat recorded. Timer reset."
- ☐ Activity log updates with the new heartbeat entry.
- ☐ If vault is in Warning status, transitions back to Active.

6.4 Redistribution

US-015: Automatic Redistribution via Crank

As a beneficiary, **I want** my inheritance to be distributed automatically when the vault triggers, **So that** I receive my share without needing to do anything.

Acceptance Criteria:

- ☐ When `current_time > last_activity + inactivity_window + grace_period`, the vault is eligible for execution.
- ☐ Any wallet (crank operator) can call `execute_redistribution`.
- ☐ Each beneficiary receives their percentage share of each asset in the vault.
- ☐ Crank operator receives the crank fee (deducted proportionally from each asset).
- ☐ Vault ATAs are closed, rent SOL returned to the vault owner.
- ☐ Vault status is set to Executed.
- ☐ All transfers happen atomically in a single transaction (or minimal number of transactions if account limits are hit).

US-016: View Redistribution Results

As a vault owner (or anyone), **I want to** see the results of a completed redistribution, **So that** I can verify that assets were distributed correctly.

Acceptance Criteria:

- ☐ Vault detail page for Executed vaults shows: execution timestamp, crank operator address, distribution details table (beneficiary name, address, token, amount received, tx signature), crank fee paid.
 - ☐ All transaction signatures link to Solana Explorer.
-

6.5 Heartbeat Monitoring

US-017: Automatic Heartbeat from Wallet Activity

As a vault owner, **I want** my normal wallet activity to automatically reset my inactivity timer, **So that** I don't need to manually check in.

Acceptance Criteria:

- ☐ When the owner signs ANY transaction on Solana, Helius sends a webhook.
 - ☐ Backend receives the webhook and submits a `record_heartbeat` to the Deadswitch program.
 - ☐ Vault's `last_activity` timestamp updates onchain.
 - ☐ Heartbeat appears in the vault's activity log.
 - ☐ Latency from owner's transaction to heartbeat recording: < 30 seconds (typical).
 - ☐ If multiple transactions happen in rapid succession, heartbeats can be batched (latest timestamp wins).
-

6.6 Notifications

US-018: Email Inactivity Alerts

As a vault owner, **I want to** receive email alerts when my inactivity timer is approaching the threshold, **So that** I can take action if I'm still alive but haven't used my wallet.

Acceptance Criteria:

- ☐ User can optionally enter an email address during vault creation or in vault settings.
 - ☐ Email is stored in the backend database (NOT onchain).
 - ☐ Alerts sent at:
 - 50% of inactivity window elapsed: subject "Deadswitch Reminder: Vault '[name]' — 50% inactivity reached"
 - 75%: subject "Deadswitch Warning: Vault '[name]' — 75% inactivity reached"
 - 90%: subject "Deadswitch Urgent: Vault '[name]' — 90% inactivity reached"
 - 100% (grace period starts): subject "Deadswitch CRITICAL: Vault '[name]' — Grace period started"
 - Daily during grace period: subject "Deadswitch CRITICAL: Vault '[name]' — [X] days until redistribution"
 - ☐ Each email contains: vault name, current inactivity duration, time remaining, link to vault detail page, "Reset Timer" CTA button (links to manual heartbeat page).
 - ☐ Unsubscribe link in every email.
 - ☐ If heartbeat is recorded (timer resets), cancel any pending alert schedule and restart.
-

6.7 Landing Page

US-019: Public Landing Page

As a visitor, I want to understand what Deadswitch is and how it works, **So that** I can decide whether to create a vault.

Acceptance Criteria:

- ☐ Hero section: headline ("Your Crypto Shouldn't Die With You"), subheadline (1 sentence explanation), CTA button ("Launch App" / "Create Your Vault").
- ☐ Problem section: 3-4 stats about lost crypto, with icons.
- ☐ How It Works section: 3-step visual (Create → Monitor → Distribute) with brief explanations.
- ☐ Comparison section: Deadswitch vs alternatives (seed phrase, multisig, Sarcophagus) — table format.
- ☐ Security section: key trust properties (non-custodial, no admin keys, open source).
- ☐ FAQ section: 5-8 most common questions.
- ☐ Footer: GitHub link, Whitepaper link, Twitter/X link, "Built on Solana" badge.

7. Information Architecture

7.1 Sitemap

```
deadswitch.xyz/
├─ / ..... Landing page (public)
├─ /app ..... Dashboard (requires wallet)
│   └─ /app/create ..... Vault creation wizard
│       └─ Step 1: Name
│           └─ Step 2: Beneficiaries
│               └─ Step 3: Assets
│                   └─ Step 4: Timing
│                       └─ Step 5: Review & Confirm
│   └─ /app/vault/[vault-id] ..... Vault detail page
│       └─ Overview tab
│           └─ Activity tab
│               └─ Settings tab
│   └─ /app/settings ..... User settings (email, notifications)
├─ /docs ..... Whitepaper (static page or PDF link)
└─ /api ..... API routes (internal)
    └─ /api/webhooks/helius ..... Helius webhook receiver
        └─ /api/vaults ..... Vault data API
            └─ /api/heartbeat ..... Manual heartbeat submission
                └─ /api/alerts ..... Alert management
```

7.2 Navigation Structure

Header (All Pages)

- Logo ("Deadswitch") — links to / or /app depending on wallet state
- Navigation: "How It Works" (scroll anchor on landing) | "Docs" | "GitHub"
- Wallet button: "Connect Wallet" / Connected address with disconnect option
- Network indicator: "Devnet" badge (for hackathon)

App Layout (Authenticated)

- Sidebar or top nav: Dashboard | Create Vault | Settings

- Breadcrumbs on detail pages: Dashboard > Vault Name

8. Page-by-Page Specifications

8.1 Landing Page (/)

Layout Structure

Header: Logo | How It Works | Docs | [Connect]

HERO SECTION

"Your Crypto Shouldn't Die With You"

Deadswitch automatically distributes your crypto to your loved ones if you become inactive. Non-custodial. No check-ins. No seed phrases to share.

[Create Your Vault] [Read Whitepaper]

STATS BAR

\$140B+ crypto lost

20% of BTC dormant

Zero tools on Solana for inheritance

HOW IT WORKS

1. SET

Create a vault, add beneficiaries

2. LIVE

Use your wallet normally. Timer auto-resets.

3. PROTECTED

If you stop, assets go to your chosen beneficiaries. Automatically.

COMPARISON TABLE

Feature	Seed Phrase	Multi sig	Sarco phagus	Dead-switch
Auto assets	X	X	X	✓
No check-in	✓	✓	X	✓
Non-custodial	✓	✓	✓	✓
No tech req	X	X	X	✓

On SolanaX X X ✓

SECURITY SECTION

- Non-custodial: we never touch your keys
- No admin backdoor: no one can drain vaults
- Open source: verify everything on GitHub
- Onchain verification: timing checks in program

FAQ SECTION (accordion)

- What happens if Deadswitch shuts down?
- Can I change my beneficiaries?
- What if I go on a long trip?
- Is this a legal will?
- What tokens are supported?

FOOTER

GitHub | Whitepaper | Twitter | Built on Solana

© 2026 Holixora | MIT License

Visual Design Direction

- **Color palette:** Dark background (#0a0a0a or #111111), primary accent in electric blue or green (#00ff88 suggested — "alive" color), red/orange for warning states.
- **Typography:** Clean, modern sans-serif (Inter or similar). Large headings, readable body text.
- **Tone:** Serious but not morbid. Professional but approachable. This is about protecting your family, not about death.
- **Animations:** Subtle — no excessive motion. Timeline animations for "How It Works" section. Smooth scroll.

8.2 Dashboard Page (/app)

Layout

Header: Logo | Dashboard | Docs | [0x7kP...3nFq]

DASHBOARD HEADER

Your Vaults [+ Create Vault]

Total Value Protected: \$12,450.00

VAULT CARDS

Family Inheritance

• Active

TVL: \$10,200

Last active: 3d ago

Triggers in: 87 days

Beneficiaries: 3

3%

Emergency Fund

⚠ Warning

TVL: \$2,250

Last active: 72d

Triggers in: 25 days

Beneficiaries: 1

80%

EMPTY STATE (if no vaults)

 No vaults yet

Create your first vault to protect your crypto assets for your loved ones.

[Create Your First Vault]

Data Display

- Progress bar under each vault card shows percentage through inactivity window.
- Color coding: green (<50%), yellow (50-75%), orange (75-90%), red (>90%).
- Relative time for "Last active" (e.g., "3 days ago", "just now").
- Countdown timer for "Triggers in" (e.g., "87 days", "2 hours 15 min").

8.3 Vault Creation Wizard (/app/create)

Step Indicator

- Name

—

○ Beneficiaries

—

○ Assets

—

○ Timing

—

○ Review

Active step is filled (●), completed steps are checkmarked (✓), future steps are empty (○).

Step 1: Name

Step 1 of 5: Name Your Vault

Vault Name *

Family Inheritance

e.g., "Family Inheritance", "Emergency Fund"

42/64

Note to Beneficiaries (optional)

To my family – this is my crypto inheritance plan. I love you all.

A message included with your vault.

78/256

[Back] [Next →]

Step 2: Beneficiaries

Step 2 of 5: Add Beneficiaries

Who should receive your assets?

#1

Name: [Wife - Sarah]

Wallet: [7xKpR...paste full address]

Share: [50] % [🗑️]

#2

Name: [Brother - James]

Wallet: [9mNq2...paste full address]

Share: [30] % [🗑️]

#3

Name: [Charity - GiveDirectly]

Wallet: [3pQw8...paste full address]

Share: [20] % [🗑️]

[+ Add Beneficiary]

Total: 100% ✓

50% 30% 20%

[← Back]

[Next →]

Step 3: Assets

Step 3 of 5: Select Assets

Choose which tokens to protect.

☒ SOL

Balance: 25.5 SOL

Amount: [10.0] [Max]

≈ \$1,500.00

☒ USDC

Balance: 5,000 USDC

Amount: [5000] [Max]

≈ \$5,000.00

o USDT Balance: 0 USDT
(No balance)

Total Vault Value: ≈ \$6,500.00

[← Back] [Next →]

Step 4: Timing

Step 4 of 5: Set Your Timing

How long should your wallet be inactive
before redistribution?

Inactivity Window:

30d

60d

90d*

180d

365d

* Recommended for most users

Grace Period:

1d

3d

7d*

14d

30d

▸ Advanced Options

Crank Fee: [0.1] % (default)

Summary:

If your wallet is inactive for
90 days + 7 day grace period =
97 days total before redistribution.

[← Back] [Next →]

Step 5: Review & Confirm

Step 5 of 5: Review & Confirm

VAULT NAME

[Edit]

Family Inheritance

"To my family – I love you all."

BENEFICIARIES

[Edit]

Sarah (50%)

7xKp...3nFq

James (30%)

9mNq...8wRt

GiveDirectly (20%)

3pQw...2mKx

ASSETS

[Edit]

10.0 SOL

≈ \$1,500.00

5,000 USDC

≈ \$5,000.00

Total: ≈ \$6,500.00

TIMING

[Edit]

Inactivity: 90 days

Grace: 7 days

Crank fee: 0.1%

Total: 97 days to redistribution

TRANSACTION COST

Gas: ~0.005 SOL (~\$0.75)

Vault rent: ~0.02 SOL (refundable)

⚠ Your assets will be locked in the vault until you cancel or redistribution occurs.

[← Back]

[🔒 Create Vault]

8.4 Vault Detail Page (/app/vault/[id])

Layout

← Dashboard / Family Inheritance

VAULT HEADER

Family Inheritance

● Active

"To my family – I love you all."

Created: Apr 14, 2026

Last Activity: 3 days ago (Apr 11, 2026)

Triggers in: 87 days (Jul 17, 2026)

PROGRESS BAR

3/90 days

▲

▲

▲

now

90 days

+7 grace

ACTION BUTTONS

[ I'm Still Here] [ Update] [ Top Up] [ Cancel]

TAB: [Overview] [Activity] [Settings]

— OVERVIEW TAB —

ASSETS

Token	Amount	Value	Per Bene.
SOL	10.0	\$1,500	see below
USDC	5,000	\$5,000	see below
Total		\$6,500	

BENEFICIARIES

Name	Address	Share	Value
Sarah	7xKp...	50%	\$3,250
James	9mNq...	30%	\$1,950
GiveDirectly	3pQw...	20%	\$1,300

— ACTIVITY TAB —

Apr 11, 14:32	Heartbeat	Swap on Jupiter tx: 5xR2... ↗
Apr 9, 08:15	Heartbeat	SOL transfer tx: 3mPq... ↗
Apr 8, 22:01	Heartbeat	NFT mint tx: 8nWk... ↗
Apr 7, 10:00	Created	Vault created tx: 2jRx... ↗

— SETTINGS TAB —

Configuration (read-only display)

Inactivity Window: 90 days

Grace Period: 7 days

Crank Fee: 0.1%

Notification Email: wira@example.com

Vault Address: DSw1...7xPq [\[Copy\]](#)

Program ID: Dead...5wRt [\[Copy\]](#)

[\[View on Solana Explorer ↗\]](#)

9. Smart Contract Specifications

9.1 Program Structure

```
programs/deadswitch/
├─ src/
│   ├── lib.rs                # Program entry point, declare_id!, module registration
│   ├── state/
│   │   ├── mod.rs
│   │   ├── vault.rs          # Vault account structure
│   │   └── config.rs          # Protocol config PDA (optional – for future protocol fee)
│   ├── instructions/
│   │   ├── mod.rs
│   │   ├── create_vault.rs    # Create vault + deposit assets
│   │   ├── update_vault.rs    # Modify vault configuration
│   │   ├── top_up_vault.rs    # Add more assets to vault
│   │   ├── cancel_vault.rs    # Cancel vault + return assets
│   │   ├── record_heartbeat.rs # Update last_activity timestamp
│   │   └── execute.rs         # Execute redistribution
│   ├── errors.rs              # Custom error codes
│   └── constants.rs           # Program constants (min/max windows, fee bounds)
├─ tests/
│   ├── create_vault.test.ts
│   ├── update_vault.test.ts
│   ├── cancel_vault.test.ts
│   ├── heartbeat.test.ts
│   ├── execute.test.ts
│   └── edge_cases.test.ts
└─ Anchor.toml
```

9.2 Constants

```
// Timing bounds (in seconds)
pub const MIN_INACTIVITY_WINDOW: i64 = 2_592_000; // 30 days
pub const MAX_INACTIVITY_WINDOW: i64 = 31_536_000; // 365 days
pub const MIN_GRACE_PERIOD: i64 = 86_400; // 1 day
pub const MAX_GRACE_PERIOD: i64 = 2_592_000; // 30 days

// Fee bounds (in basis points)
pub const MIN_CRANK_FEE_BPS: u16 = 1; // 0.01%
pub const MAX_CRANK_FEE_BPS: u16 = 500; // 5%
pub const DEFAULT_CRANK_FEE_BPS: u16 = 10; // 0.1%

// Limits
pub const MAX_BENEFICIARIES: usize = 10;
pub const MAX_ASSETS: usize = 20;
pub const MAX_VAULT_NAME_LEN: usize = 64;
pub const MAX_NOTE_LEN: usize = 256;
pub const MAX_BENEFICIARY_NAME_LEN: usize = 32;

// Shares
```

```

pub const TOTAL_SHARE_BPS: u16 = 10_000;           // 100%

// Clock tolerance
pub const CLOCK_DRIFT_TOLERANCE: i64 = 30;         // 30 seconds

// PDA seeds
pub const VAULT_SEED: &[u8] = b"vault";

```

9.3 Error Codes

```

#[error_code]
pub enum DeadswitchError {
    #[msg("Inactivity window must be between 30 and 365 days")]
    InvalidInactivityWindow,

    #[msg("Grace period must be between 1 and 30 days")]
    InvalidGracePeriod,

    #[msg("Crank fee must be between 0.01% and 5%")]
    InvalidCrankFee,

    #[msg("Beneficiary shares must total exactly 100%")]
    SharesNotOneHundredPercent,

    #[msg("Too many beneficiaries (max 10)")]
    TooManyBeneficiaries,

    #[msg("At least one beneficiary is required")]
    NoBeneficiaries,

    #[msg("Too many assets (max 20)")]
    TooManyAssets,

    #[msg("At least one asset is required")]
    NoAssets,

    #[msg("Vault name is too long (max 64 characters)")]
    VaultNameTooLong,

    #[msg("Note is too long (max 256 characters)")]
    NoteTooLong,

    #[msg("Beneficiary name is too long (max 32 characters)")]
    BeneficiaryNameTooLong,

    #[msg("Cannot use your own address as a beneficiary")]
    SelfBeneficiary,

    #[msg("Duplicate beneficiary address")]
    DuplicateBeneficiary,

    #[msg("Invalid beneficiary address")]

```

```

InvalidBeneficiary,

#[msg("Vault is not in a modifiable state")]
VaultNotModifiable,

#[msg("Vault is not eligible for execution")]
VaultNotTriggered,

#[msg("Heartbeat timestamp is in the future")]
FutureHeartbeat,

#[msg("Heartbeat timestamp is not newer than last activity")]
StaleHeartbeat,

#[msg("Unauthorized – only the vault owner can perform this action")]
Unauthorized,

#[msg("Unauthorized – invalid heartbeat authority")]
UnauthorizedHeartbeat,

#[msg("Insufficient deposit amount")]
InsufficientDeposit,

#[msg("Arithmetic overflow")]
ArithmeticOverflow,

#[msg("Beneficiary share is zero")]
ZeroShare,
}

```

9.4 Test Cases

create_vault tests

#	Test Case	Expected
TC-001	Create vault with valid params (1 beneficiary, SOL only)	Success, vault PDA created, SOL deposited
TC-002	Create vault with 3 beneficiaries and 2 assets (SOL + USDC)	Success
TC-003	Create vault with 10 beneficiaries (max)	Success
TC-004	Create vault with 11 beneficiaries	Fail: TooManyBeneficiaries
TC-005	Create vault with 0 beneficiaries	Fail: NoBeneficiaries
TC-006	Shares total 99%	Fail: SharesNotOneHundredPercent
TC-007	Shares total 101%	Fail: SharesNotOneHundredPercent
TC-008	Inactivity window = 29 days	Fail: InvalidInactivityWindow
TC-009	Inactivity window = 366 days	Fail: InvalidInactivityWindow
TC-010	Grace period = 0	Fail: InvalidGracePeriod

TC-011	Self as beneficiary	Fail: SelfBeneficiary
TC-012	Duplicate beneficiary addresses	Fail: DuplicateBeneficiary
TC-013	Crank fee = 0	Fail: InvalidCrankFee
TC-014	Crank fee = 501 bps	Fail: InvalidCrankFee
TC-015	Name > 64 chars	Fail: VaultNameTooLong
TC-016	Deposit 0 SOL	Fail: InsufficientDeposit
TC-017	Deposit more SOL than wallet balance	Fail: insufficient funds (Solana error)
TC-018	Verify last_activity = current timestamp after creation	Success
TC-019	Verify PDA derivation is correct	Success
TC-020	Create second vault for same owner (vault_id increments)	Success

record_heartbeat tests

#	Test Case	Expected
TC-021	Record heartbeat on active vault	Success, last_activity updated
TC-022	Record heartbeat on warning vault	Success, status back to Active
TC-023	Record heartbeat on cancelled vault	Fail: VaultNotModifiable
TC-024	Record heartbeat on executed vault	Fail: VaultNotModifiable
TC-025	Record heartbeat with timestamp in the future	Fail: FutureHeartbeat
TC-026	Record heartbeat with older timestamp than last_activity	Fail: StaleHeartbeat
TC-027	Record heartbeat by unauthorized wallet	Fail: UnauthorizedHeartbeat
TC-028	Record heartbeat by vault owner directly	Success

execute_redistribution tests

#	Test Case	Expected
TC-029	Execute before inactivity window elapsed	Fail: VaultNotTriggered
TC-030	Execute during grace period	Fail: VaultNotTriggered
TC-031	Execute after window + grace period (SOL only, 1 beneficiary)	Success, SOL transferred
TC-032	Execute with 3 beneficiaries, 50/30/20 split, SOL + USDC	Success, correct amounts
TC-033	Verify crank fee deducted correctly	Success, crank gets fee
TC-034	Verify vault ATAs closed after execution	Success, rent returned
TC-035	Execute already-executed vault	Fail: VaultNotModifiable
TC-036	Execute cancelled vault	Fail: VaultNotModifiable

TC-037	Anyone can call execute (permissionless)	Success
TC-038	Verify exact amounts with rounding (dust goes to last beneficiary)	Success

cancel_vault tests

#	Test Case	Expected
TC-039	Cancel active vault	Success, assets returned, ATAs closed
TC-040	Cancel warning vault	Success
TC-041	Cancel executed vault	Fail: VaultNotModifiable
TC-042	Cancel by non-owner	Fail: Unauthorized
TC-043	Verify all SOL returned to owner	Success
TC-044	Verify all USDC returned to owner	Success
TC-045	Verify vault status = Cancelled	Success

update_vault tests

#	Test Case	Expected
TC-046	Update beneficiary list	Success, last_activity reset
TC-047	Update inactivity window	Success
TC-048	Update by non-owner	Fail: Unauthorized
TC-049	Update executed vault	Fail: VaultNotModifiable
TC-050	Update with invalid shares (not 100%)	Fail: SharesNotOneHundredPercent

10. Backend Specifications

10.1 Backend Architecture

```
src/
├─ app/
│   └─ api/
│       ├── webhooks/
│       │   └─ helius/
│       │       └─ route.ts           # POST – Helius webhook receiver
│       ├── vaults/
│       │   ├── route.ts             # GET – List vaults for connected wallet
│       │   └─ [id]/
│       │       └─ route.ts          # GET – Vault detail with activity log
│       ├── heartbeat/
│       │   └─ route.ts              # POST – Manual heartbeat submission
│       ├── alerts/
│       │   └─ route.ts              # PUT – Update alert preferences
```

```

| | | └─ prices/
| | |   └─ route.ts          # GET – Token prices (cached)
| | └─ (landing)/
| |   └─ page.tsx           # Landing page
| └─ app/
| |   └─ layout.tsx         # App layout with wallet provider
| |   └─ page.tsx           # Dashboard
| |   └─ create/
| |     └─ page.tsx         # Vault creation wizard
| |   └─ vault/
| |     └─ [id]/
| |       └─ page.tsx       # Vault detail
| |   └─ settings/
| |     └─ page.tsx         # User settings
| └─ layout.tsx             # Root layout
└─ components/
  └─ ui/                    # Shadcn/UI components
  └─ wallet/                # Wallet connection components
  └─ vault/                 # Vault-specific components
    └─ VaultCard.tsx
    └─ VaultWizard.tsx
    └─ BeneficiaryForm.tsx
    └─ AssetSelector.tsx
    └─ TimingSelector.tsx
    └─ VaultDetail.tsx
    └─ ActivityLog.tsx
    └─ ProgressBar.tsx
  └─ landing/               # Landing page sections
└─ lib/
  └─ solana/
    └─ program.ts           # Anchor program client
    └─ instructions.ts     # Transaction builders
    └─ pdas.ts             # PDA derivation helpers
  └─ helius/
    └─ webhook.ts           # Webhook registration & validation
    └─ types.ts            # Helius webhook payload types
  └─ db/
    └─ schema.ts           # Drizzle schema
    └─ index.ts            # DB connection
    └─ queries.ts          # Query helpers
  └─ email/
    └─ send.ts             # Email sending (Resend)
    └─ templates.ts        # Email templates
  └─ prices.ts             # Token price fetching + caching
  └─ constants.ts          # Shared constants
└─ hooks/
  └─ useVaults.ts          # Fetch user vaults
  └─ useVaultDetail.ts     # Fetch single vault detail
  └─ useTokenBalances.ts   # Fetch wallet token balances
  └─ useTokenPrices.ts     # Fetch token prices
└─ types/
  └─ index.ts              # Shared TypeScript types

```

10.2 Helius Webhook Receiver

Endpoint: `POST /api/webhooks/helius`

Purpose: Receives webhook callbacks from Helius when a monitored wallet (vault owner) signs a transaction.

Flow:

1. Receive POST request from Helius.
2. Validate webhook signature (Helius signs webhooks with a shared secret).
3. Extract the wallet address and transaction timestamp from the payload.
4. Look up which vault(s) this wallet owns.
5. For each active vault, submit a `record_heartbeat` transaction to the Solana program.
6. Log the heartbeat in the database.
7. Reset any pending alert schedules for the vault.
8. Return 200 OK.

Validation:

- Verify `Authorization` header matches the configured Helius webhook secret.
- Reject requests with invalid or missing auth.
- Idempotency: if the same transaction signature has already been processed, skip (dedup via database).
- Rate limit: max 100 requests per second per wallet (Helius may batch).

Error handling:

- If heartbeat transaction fails (e.g., vault already cancelled), log the error but return 200 (so Helius doesn't retry).
- If database write fails, log error, still return 200 (heartbeat is onchain, DB is cache).

11. API Specifications

11.1 Internal API Routes

All API routes are internal (used by the frontend). No public API in MVP.

GET `/api/vaults?owner={publicKey}`

Returns all vaults for a given owner, with computed status and metadata.

Response:

```
{
  "vaults": [
    {
      "vaultId": "1",
      "publicKey": "DSw1...7xPq",
      "name": "Family Inheritance",
      "status": "active",
      "totalValueUsd": 6500.00,
      "lastActivity": "2026-04-11T14:32:00Z",
      "triggersAt": "2026-07-17T14:32:00Z",
      "progressPercent": 3.3,
      "beneficiaryCount": 3,
      "assets": [
        { "mint": "So1111111111111111111111111111111111111111", "symbol": "SOL", "amount": "10000000000",
```

```

"decimals": 9, "valueUsd": 1500.00 },
    { "mint": "EPjFWdd5AufqSSqeM2qN1xzybapC8G4wEGGkZwyTDt1v", "symbol": "USDC", "amount":
"5000000000", "decimals": 6, "valueUsd": 5000.00 }
  ]
}
]
}

```

GET /api/vaults/{vaultPublicKey}

Returns full vault detail including beneficiaries, assets, and activity log.

Response:

```

{
  "vault": {
    "vaultId": "1",
    "publicKey": "DSw1...7xPq",
    "owner": "7xKp...3nFq",
    "name": "Family Inheritance",
    "note": "To my family – I love you all.",
    "status": "active",
    "inactivityWindowDays": 90,
    "gracePeriodDays": 7,
    "crankFeeBps": 10,
    "lastActivity": "2026-04-11T14:32:00Z",
    "createdAt": "2026-04-07T10:00:00Z",
    "triggersAt": "2026-07-17T14:32:00Z",
    "progressPercent": 3.3,
    "assets": [...],
    "beneficiaries": [
      { "wallet": "7xKp...3nFq", "name": "Sarah", "shareBps": 5000, "estimatedValueUsd": 3250.00 },
      { "wallet": "9mNq...8wRt", "name": "James", "shareBps": 3000, "estimatedValueUsd": 1950.00 },
      { "wallet": "3pQw...2mKx", "name": "GiveDirectly", "shareBps": 2000, "estimatedValueUsd": 1300.00 }
    ],
    "activityLog": [
      { "timestamp": "2026-04-11T14:32:00Z", "type": "heartbeat", "txSignature": "5xR2...",
"description": "Swap on Jupiter" },
      { "timestamp": "2026-04-07T10:00:00Z", "type": "created", "txSignature": "2jRx...", "description":
"Vault created" }
    ]
  }
}

```

POST /api/heartbeat

Manually record a heartbeat (frontend submits the signed transaction, backend confirms onchain).

Request:

```

{
  "vaultPublicKey": "DSw1...7xPq",

```

```
"txSignature": "5xR2..."
}
```

PUT /api/alerts

Update alert preferences for a vault.

Request:

```
{
  "vaultPublicKey": "DSw1...7xPq",
  "email": "wira@example.com",
  "enabled": true
}
```

GET /api/prices

Returns cached token prices in USD.

Response:

```
{
  "prices": {
    "SOL": 150.00,
    "USDC": 1.00,
    "USDT": 1.00
  },
  "updatedAt": "2026-04-14T10:00:00Z"
}
```

12. Database Schema

12.1 Tables (PostgreSQL via Drizzle ORM)

vaults — Cached vault metadata (source of truth is onchain)

```
CREATE TABLE vaults (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  vault_pubkey VARCHAR(44) UNIQUE NOT NULL,      -- Onchain vault PDA
  owner_pubkey VARCHAR(44) NOT NULL,             -- Owner wallet
  vault_id_onchain BIGINT NOT NULL,              -- Onchain vault_id (for PDA derivation)
  name        VARCHAR(64) NOT NULL,
  note        VARCHAR(256),
  status      VARCHAR(20) NOT NULL DEFAULT 'active', -- active, warning, triggered, executed,
  cancelled
  inactivity_window_secs BIGINT NOT NULL,
  grace_period_secs      BIGINT NOT NULL,
  crank_fee_bps          SMALLINT NOT NULL,
  last_activity          TIMESTAMP WITH TIME ZONE NOT NULL,
  created_at             TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  updated_at             TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
)
```

```

    executed_at    TIMESTAMP WITH TIME ZONE,
    cancelled_at   TIMESTAMP WITH TIME ZONE
);

CREATE INDEX idx_vaults_owner ON vaults(owner_pubkey);
CREATE INDEX idx_vaults_status ON vaults(status);

```

vault_assets — Assets in each vault

```

CREATE TABLE vault_assets (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  vault_id    UUID NOT NULL REFERENCES vaults(id) ON DELETE CASCADE,
  mint        VARCHAR(44) NOT NULL,           -- Token mint address
  symbol       VARCHAR(10) NOT NULL,          -- SOL, USDC, USDT
  amount       BIGINT NOT NULL,               -- Raw amount (lamports/smallest unit)
  decimals     SMALLINT NOT NULL,
  created_at  TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_vault_assets_vault ON vault_assets(vault_id);

```

vault_beneficiaries — Beneficiaries for each vault

```

CREATE TABLE vault_beneficiaries (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  vault_id    UUID NOT NULL REFERENCES vaults(id) ON DELETE CASCADE,
  wallet       VARCHAR(44) NOT NULL,
  name        VARCHAR(32) NOT NULL,
  share_bps   SMALLINT NOT NULL,
  created_at  TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_vault_beneficiaries_vault ON vault_beneficiaries(vault_id);

```

heartbeats — Heartbeat log (activity history)

```

CREATE TABLE heartbeats (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  vault_id    UUID NOT NULL REFERENCES vaults(id) ON DELETE CASCADE,
  tx_signature VARCHAR(88) UNIQUE NOT NULL,   -- Solana tx that triggered this heartbeat
  source_tx    VARCHAR(88),                  -- Original owner tx that Helius detected
  activity_type VARCHAR(20) NOT NULL,         -- heartbeat, manual, creation, update, top_up,
  execution, cancellation
  description  VARCHAR(128),
  recorded_at  TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_heartbeats_vault ON heartbeats(vault_id);
CREATE INDEX idx_heartbeats_source_tx ON heartbeats(source_tx);

```

alert_configs — Alert preferences per vault

```
CREATE TABLE alert_configs (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  vault_id    UUID NOT NULL REFERENCES vaults(id) ON DELETE CASCADE UNIQUE,  
  email       VARCHAR(255),  
  telegram_id VARCHAR(64),  
  enabled     BOOLEAN NOT NULL DEFAULT true,  
  last_alert_sent_at TIMESTAMP WITH TIME ZONE,  
  last_alert_threshold SMALLINT,          -- 50, 75, 90, 100  
  created_at  TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),  
  updated_at  TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()  
);  
  
CREATE INDEX idx_alert_configs_vault ON alert_configs(vault_id);
```

processed_webhooks — Idempotency for Helius webhooks

```
CREATE TABLE processed_webhooks (  
  tx_signature  VARCHAR(88) PRIMARY KEY,  
  processed_at  TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()  
);  
  
-- Auto-clean entries older than 7 days (via cron or app logic)
```

13. Notification System

13.1 Alert Engine

A background job (cron or setInterval) runs every hour:

1. Query all active vaults from the database.
2. For each vault, compute `elapsed_percent = (now - last_activity) / inactivity_window * 100`.
3. Determine which alert threshold has been crossed (50%, 75%, 90%, 100%).
4. If the threshold is higher than `last_alert_threshold`, send the alert and update `last_alert_threshold`.
5. During grace period (100%), send daily alerts.

13.2 Email Templates

Provider: Resend (free tier: 100 emails/day — sufficient for MVP).

50% Alert

Subject: Deadswitch Reminder: Vault "Family Inheritance" – 45 days inactive

Hi,

Your Deadswitch vault "Family Inheritance" has been inactive for 45 days (50% of your 90-day window).

If you're still active, simply use your Solana wallet or click the button below to reset your timer.

[Reset My Timer →]

Time remaining: 45 days + 7 day grace period

– Deadswitch

90% Alert

Subject: ⚠ URGENT – Vault "Family Inheritance" – 81 days inactive

Your vault is 90% through its inactivity window.

If you do NOT perform any wallet activity within the next 9 days + 7 day grace period, your assets will be redistributed to your beneficiaries.

If you are still active, please use your wallet or click below immediately.

[Reset My Timer NOW →]

– Deadswitch

14. Crank Bot Specifications

14.1 MVP Crank Bot

A simple TypeScript script that runs as a background process.

Behavior:

1. Every 60 seconds, query all vaults from the Deadswitch program (or from the database cache).
2. Filter vaults where `current_time > last_activity + inactivity_window + grace_period`.
3. For each eligible vault, submit an `execute_redistribution` transaction.
4. Log results (success/failure) to console and database.

```
crank/
├─ src/
│   ├─ index.ts           # Main loop
│   ├─ scanner.ts         # Scan for triggered vaults
│   ├─ executor.ts        # Submit execution transactions
│   └─ config.ts          # Environment config
├─ package.json
└─ tsconfig.json
```

14.2 Crank Bot Configuration

Env Variable	Description	Default
SOLANA_RPC_URL	Solana RPC endpoint	Helius devnet RPC

CRANK_WALLET_PRIVATE_KEY	Crank operator wallet (pays gas, receives fees)	Required
DEADSWITCH_PROGRAM_ID	Deadswitch program address	Required
SCAN_INTERVAL_MS	How often to scan (ms)	60000 (1 min)
DATABASE_URL	PostgreSQL connection (for vault cache)	Required
MIN_PROFIT_LAMPORTS	Minimum crank fee to bother executing	10000 (0.00001 SOL)

15. Non-Functional Requirements

15.1 Performance

Metric	Target
Landing page load (LCP)	< 2.5 seconds
App page load (LCP)	< 3.0 seconds
Vault creation wizard step transitions	< 200ms
Transaction confirmation feedback	< 500ms after confirmation
Webhook processing latency (Helius → heartbeat onchain)	< 30 seconds
API response time (vault list, detail)	< 500ms
Crank scan + execute cycle	< 5 seconds per vault

15.2 Reliability

Metric	Target
Frontend uptime	99.5% (Vercel SLA)
Backend/webhook uptime	99% (acceptable for MVP)
Anchor program	100% (onchain — always available if Solana is up)
False trigger rate	0% (enforced by onchain timestamp checks)

15.3 Security

Requirement	Implementation
No admin drain capability	No instruction in the program allows arbitrary fund withdrawal
Webhook authentication	Helius webhook signature verification
CSRF protection	Next.js built-in CSRF for API routes
Rate limiting	Basic rate limiting on API routes (100 req/min per IP)
Input validation	All user input validated on frontend AND in Anchor program

No PII onchain	Email, Telegram stored in DB only, not in vault accounts
HTTPS only	Enforced by Vercel
Environment secrets	All keys in environment variables, never committed

15.4 Accessibility

Requirement	Target
WCAG level	2.1 AA
Keyboard navigation	Full keyboard support for wizard and dashboard
Screen reader	ARIA labels on all interactive elements
Color contrast	4.5:1 minimum contrast ratio
Focus indicators	Visible focus rings on all interactive elements
Motion	Respect prefers-reduced-motion

15.5 Browser Support

Browser	Version
Chrome	Latest 2 versions
Firefox	Latest 2 versions
Safari	Latest 2 versions
Edge	Latest 2 versions
Mobile Chrome	Latest
Mobile Safari	Latest

16. Tech Stack & Architecture Decisions

16.1 Final Tech Stack

Layer	Technology	Rationale
Smart Contract	Anchor (Rust)	Industry standard for Solana programs. Type safety. Testing framework built-in.
Frontend Framework	Next.js 15 (App Router)	SSR for landing page SEO, API routes for backend, React 19 for UI. Single deployment.
Language	TypeScript 5.x (strict)	Type safety across frontend + backend.
Styling	TailwindCSS 4 + shadcn/ui	Rapid development, consistent design, accessible components.
Wallet	@solana/wallet-adapter-react	Standard Solana wallet integration. Supports all major wallets.

Solana Client	@coral-xyz/anchor + @solana/web3.js	Program interaction, transaction building.
Database	PostgreSQL (Neon serverless)	Free tier sufficient for MVP. Drizzle ORM for type-safe queries.
ORM	Drizzle ORM	Lightweight, TypeScript-native, SQL-like query builder.
Email	Resend	Developer-friendly, free tier (100/day), React email templates.
Monitoring	Helius Webhooks	Wallet activity monitoring. Free tier: 10 webhook addresses.
Hosting	Vercel (frontend + API)	Free tier, instant deploys, edge network.
Crank Bot	Node.js script (PM2 on VPS or local)	Simple, reliable, easy to deploy.
Testing	Vitest (frontend) + Anchor test (program)	Fast, modern, TypeScript-native.
Linting	ESLint 9 (flat config) + Prettier	Code consistency.

16.2 Architecture Decision Records

ADR-001: Escrow vs Approval Model

Decision: Escrow (assets deposited into vault PDAs). **Rationale:** Guarantees funds are available at redistribution time. Approval model risks owner spending assets before trigger, leaving beneficiaries with nothing. For inheritance — reliability > flexibility.

ADR-002: Monorepo vs Separate Repos

Decision: Monorepo with Anchor + Next.js in one repo. **Rationale:** Easier to keep program IDL and frontend types in sync. Single CI/CD pipeline. Standard for hackathon projects.

ADR-003: Onchain Status vs Computed Status

Decision: Compute status from timestamps (Active/Warning/Triggered are derived, only Executed/Cancelled are stored). **Rationale:** No need for state-transition transactions. Status is always accurate based on current time vs last_activity. Saves gas and complexity.

ADR-004: Heartbeat Authority vs Owner-Only Heartbeats

Decision: Dedicated heartbeat authority wallet (backend) + owner can also heartbeat directly. **Rationale:** Automated monitoring should be seamless (user doesn't sign heartbeats). Backend watches Helius and records heartbeats on behalf of owners. Owner can always override manually.

ADR-005: Database as Cache, Not Source of Truth

Decision: All critical state (vault config, assets, timing) lives onchain. Database is a read cache + notification state. **Rationale:** If the database is lost, the protocol still works. Vaults can be reconstructed by reading onchain state. Database only stores non-critical data (email preferences, webhook dedup, activity descriptions).

17. Environment & Configuration

17.1 Environment Variables

Application (.env.local)

```
# Solana
NEXT_PUBLIC_SOLANA_NETWORK=devnet
NEXT_PUBLIC_SOLANA_RPC_URL=https://devnet.helius-rpc.com/?api-key=YOUR_KEY
NEXT_PUBLIC_DEADSWITCH_PROGRAM_ID=DeadSwitch...programId

# Helius
HELIUS_API_KEY=your-helius-api-key
HELIUS_WEBHOOK_SECRET=your-webhook-secret

# Database
DATABASE_URL=postgresql://user:pass@host:5432/deadswitch

# Email
RESEND_API_KEY=re_XXXXXXXXXXXX
ALERT_FROM_EMAIL=alerts@deadswitch.xyz

# Heartbeat Authority
HEARTBEAT_AUTHORITY_PRIVATE_KEY=base58-encoded-private-key

# App
NEXT_PUBLIC_APP_URL=https://deadswitch.xyz
```

Crank Bot (.env)

```
SOLANA_RPC_URL=https://devnet.helius-rpc.com/?api-key=YOUR_KEY
CRANK_WALLET_PRIVATE_KEY=base58-encoded-private-key
DEADSWITCH_PROGRAM_ID=DeadSwitch...programId
DATABASE_URL=postgresql://user:pass@host:5432/deadswitch
SCAN_INTERVAL_MS=60000
MIN_PROFIT_LAMPORTS=10000
```

18. Third-Party Integrations

18.1 Helius

Feature Used	Purpose	Tier
RPC Node	Solana RPC for all onchain reads/writes	Free (devnet)
Enhanced Webhooks	Monitor vault owner wallets for activity	Free (10 addresses)
DAS API	(Future) Asset queries for NFT support	Free tier

Limitation: Free tier supports 10 webhook addresses. For MVP this is fine. Production will need paid tier (\$0 for first 1M credits).

Webhook registration flow:

- On vault creation, call Helius API to register a webhook for the owner's wallet.
- Webhook type: `ENHANCED` (provides parsed transaction data).
- Transaction types to monitor: `ANY` (we care about ANY signed transaction).
- Webhook URL: `https://deadswitch.xyz/api/webhooks/helius`.

18.2 Resend (Email)

Feature	Purpose	Tier
Transactional email	Inactivity alert emails	Free (100/day)
React Email templates	Styled alert emails	Included

18.3 Neon (PostgreSQL)

Feature	Purpose	Tier
Serverless PostgreSQL	Vault cache, alerts, heartbeat log	Free (0.5 GB storage)

18.4 Vercel

Feature	Purpose	Tier
Hosting	Frontend + API routes	Free (Hobby)
Edge network	Global CDN for landing page	Included
Serverless functions	API route execution	Included

19. Error Handling & Edge Cases

19.1 Frontend Error States

Scenario	User-Facing Message	Recovery Action
Wallet not connected	"Connect your wallet to continue"	Show connect button
Insufficient SOL for gas	"You need at least 0.05 SOL for transaction fees"	Show SOL balance, link to faucet (devnet)
Transaction rejected by user	"Transaction was cancelled"	"Try Again" button
Transaction failed (program error)	"Transaction failed: [error message]"	Show error details, "Try Again" button
Network error (RPC down)	"Unable to connect to Solana. Please try again."	Auto-retry with exponential backoff, manual retry button
Vault not found	"This vault doesn't exist or has been removed"	Link back to dashboard
Unauthorized (not vault owner)	"You don't have permission to manage this vault"	Show vault in read-only mode
Helius webhook registration failed	(Silent) — vault still works, just no auto-heartbeats	Log error, alert admin, user can use manual heartbeat

19.2 Smart Contract Edge Cases

Edge Case	Handling
Rounding dust in splits	Remainder lamports go to the last beneficiary in the list. This ensures no dust is left in the vault. Example: 100 lamports split 33%/33%/34% = 33 + 33 + 34.
Vault ATA doesn't exist for beneficiary	The <code>execute_redistribution</code> instruction creates ATAs for beneficiaries if they don't exist (using <code>init_if_needed</code> , rent paid from vault funds).
SOL handling	Native SOL is wrapped to wSOL for vault storage (SPL token operations). On redistribution, wSOL is unwrapped back to native SOL for beneficiaries via CPI to token program + close account.
Clock drift	Allow 30-second tolerance for heartbeat timestamps vs onchain clock.
Owner creates vault with very small amounts	Allow it — no minimum. The crank fee might exceed the profit for cranks, but the protocol still works.
All beneficiary wallets are closed/invalid	ATA <code>init_if_needed</code> handles this. If a wallet truly cannot receive tokens (e.g., it's a program address that can't hold tokens), the transaction will fail. This is a configuration error by the vault owner.
Token account with 0 balance at execution time	Skip tokens with 0 balance during redistribution (no transfer needed).
Multiple vaults for same owner	Each vault has independent settings and monitoring. Helius webhook covers the wallet (not the vault), so one webhook handles all vaults for an owner.

20. Testing Strategy

20.1 Testing Layers

Layer	Tool	Focus
Anchor Program	<code>anchor test</code> (Mocha/Chai)	All instructions, error cases, state transitions, math correctness
Frontend Components	Vitest + React Testing Library	Component rendering, form validation, state management
API Routes	Vitest	Webhook processing, vault queries, heartbeat logic
Integration	Manual + Devnet	End-to-end flow on devnet (create → heartbeat → execute)
Visual	Manual browser testing	Responsive layout, wallet flows, loading states

20.2 Test Coverage Targets

Area	Target
Anchor program instructions	100% instruction coverage, all error paths
Frontend form validation	All validation rules tested
API webhook processing	Happy path + auth failure + dedup

E2E devnet flow	Manual test checklist (see below)
-----------------	-----------------------------------

20.3 Manual E2E Test Checklist (Devnet)

- ☐ Connect Phantom wallet on devnet
- ☐ Airdrop devnet SOL
- ☐ Create vault with 1 beneficiary, SOL only
- ☐ Verify vault appears on dashboard with correct status
- ☐ View vault detail page — all data correct
- ☐ Perform a devnet transaction — heartbeat appears in activity log
- ☐ Update vault beneficiary percentage
- ☐ Top up vault with more SOL
- ☐ Cancel vault — assets returned to wallet
- ☐ Create vault with 3 beneficiaries, SOL + USDC
- ☐ Fast-forward time (devnet program with debug time override) to trigger redistribution
- ☐ Run crank bot — redistribution executes
- ☐ Verify all beneficiaries received correct amounts
- ☐ Verify crank operator received fee
- ☐ Verify vault shows "Executed" status with details

21. Deployment Strategy

21.1 Anchor Program

Step	Command	Environment
Build	<code>anchor build</code>	Local
Test	<code>anchor test</code>	Local (localnet)
Deploy to devnet	<code>anchor deploy --provider.cluster devnet</code>	Devnet
Verify IDL	<code>anchor idl init --filepath target/idl/deadswitch.json <PROGRAM_ID></code>	Devnet

21.2 Frontend + Backend

Step	Method	Environment
Development	<code>npm run dev</code>	Local (localhost:3000)
Preview	Vercel preview deploy (PR)	Vercel
Production	Vercel auto-deploy (main branch)	Vercel

21.3 Crank Bot

Step	Method	Environment
Development	<code>npx ts-node src/index.ts</code>	Local

Production	PM2 on VPS (or local machine for MVP)	VPS / Local
------------	---------------------------------------	-------------

21.4 Database

Step	Method	Environment
Schema push	<code>npx drizzle-kit push</code>	Neon
Migrations	<code>npx drizzle-kit generate + npx drizzle-kit migrate</code>	Neon

22. Analytics & Metrics

22.1 Key Metrics to Track (Post-MVP)

Metric	How to Measure
Vaults created (cumulative)	Database count
Active vaults	Database count where status = active
Total Value Locked (TVL)	Sum of onchain vault balances × token prices
Heartbeats recorded (daily)	Database count per day
Redistributions executed	Database count where status = executed
Average vault size (USD)	TVL / active vaults
Average inactivity window chosen	<code>AVG(inactivity_window_secs)</code>
Vault cancellation rate	Cancelled / (Created)
Alert email open rate	Resend analytics
Landing page → Vault creation conversion	Vercel analytics

22.2 MVP Analytics

For hackathon MVP, keep it simple:

- Vercel Analytics (built-in, free) for page views and web vitals.
 - Console logging for webhook processing and crank execution.
 - Database queries for vault counts and statuses.
-

23. Development Phases & Milestones

Phase 1: Smart Contract (April 14-24, 2026) — 10 days

Day	Task	Deliverable
1-2	Project setup: Anchor init, account structures, constants, errors	Compilable program skeleton

3-4	create_vault + tests	Working vault creation with deposits
5	record_heartbeat + tests	Working heartbeat recording
6	cancel_vault + tests	Working cancellation with asset return
7-8	execute_redistribution + tests	Working redistribution with correct splits
9	update_vault + top_up_vault + tests	Working vault modification
10	Edge cases, full test suite, devnet deploy	Program deployed to devnet

Phase 2: Backend + Integrations (April 25-30, 2026) — 6 days

Day	Task	Deliverable
11	Next.js project setup, DB schema, Drizzle config	App scaffold
12	Helius webhook integration (register + receive + process)	Auto-heartbeat working
13	Vault API routes (list, detail, heartbeat)	API endpoints working
14	Alert engine (cron job, email templates, Resend)	Email alerts working
15	Crank bot (scanner + executor)	Crank bot executing triggered vaults
16	Integration testing on devnet	Full flow validated

Phase 3: Frontend (May 1-5, 2026) — 5 days

Day	Task	Deliverable
17	Landing page (hero, how it works, comparison, FAQ)	Landing page live
18	Wallet connect + Dashboard	Dashboard showing vaults
19	Vault creation wizard (Steps 1-5)	Full creation flow
20	Vault detail page (overview, activity, settings tabs)	Detail page with live data
21	Update, top-up, cancel flows + manual heartbeat	All vault management actions

Phase 4: Polish & Submission (May 6-11, 2026) — 6 days

Day	Task	Deliverable
22	UI polish, animations, loading states, error states	Polished UI
23	Mobile responsiveness testing + fixes	Mobile-ready
24	Demo video recording (3-5 minutes)	Demo video
25	README, documentation, code cleanup	Clean repo
26	Hackathon registration (deadline May 4 — do this on May 1-2!)	Registered
27	Final testing, submission	Submitted

Critical: Register on colosseum.com by May 4, 2026!

24. Risk Register

#	Risk	Probability	Impact	Mitigation
R1	Anchor program takes longer than 10 days	Medium	High	Start with minimal instructions (create + execute), add update/top-up as stretch
R2	Helius free tier webhook limit (10 addresses)	Low (MVP)	Low	Sufficient for demo. Upgrade to paid for production
R3	SOL wrapping/unwrapping complexity	Medium	Medium	Test early. If too complex, MVP can be USDC-only (skip SOL wrapping)
R4	Remaining account space for beneficiaries/assets on Vault PDA	Medium	Medium	Use realloc if needed, or fixed max size with padding
R5	Transaction size limit for execute_redistribution (many beneficiaries)	Low	Medium	Cap at 5 beneficiaries for MVP if needed. Or split into multiple transactions
R6	Demo video quality	Low	Medium	Script and rehearse. Record multiple takes. Use OBS + screen recording
R7	Devnet instability during demo	Low	Medium	Record demo video well in advance. Have backup screenshots
R8	Time management (solo developer)	High	High	Follow phase plan strictly. Cut Should-Have features if behind schedule
R9	Forgetting to register by May 4	Low	Critical	Set calendar reminder for May 1-2

25. Open Questions & Decisions

#	Question	Options	Decision	Status
Q1	Should vault accounts use fixed-size arrays or Vec for beneficiaries?	Fixed (simpler, predictable rent) vs Vec (flexible, realloc needed)	Fixed-size array with MAX_BENEFICIARIES = 10 — predictable account size, no realloc	Decided
Q2	Should SOL be wrapped to wSOL in the vault, or handled natively?	wSOL (uniform SPL handling) vs native (simpler for users, complex for program)	wSOL — uniform token handling, auto-unwrap on redistribution	Decided
Q3	How to handle devnet time simulation for demo?	Custom clock instruction (debug only) vs very short windows (5 min) for demo	Short windows for demo (5-minute inactivity, 1-minute grace) with a separate "demo mode" flag	Decided
Q4	Should the protocol fee be included in MVP?	Yes (shows business model) vs No (simplify)	No protocol fee in MVP — show it in the business plan, implement post-hackathon	Decided
Q5	Domain name for the project?	deadswitch.xyz, deadswitch.io, deadswitch.app	TBD — check availability	Open

Q6	Should beneficiaries be notified before redistribution?	Yes (email) vs No (MVP simplicity)	No for MVP — add in post-hackathon phase	Decided
Q7	Git repository name and organization?	holixora/deadswitch, me-workspace/deadswitch	TBD — align with hackathon submission	Open

```
└─ deadswitch.json          # Generated IDL
└─ migrations/              # Anchor migrations
└─ Anchor.toml
└─ Cargo.toml
└─ package.json             # Root workspace
└─ README.md
└─ LICENSE                  # MIT
└─ .gitignore
└─ .env.example
└─ docs/
    └─ WHITEPAPER.md
    └─ WHITEPAPER.pdf
    └─ PRD.md
```

26.2 Token Mint Addresses (Devnet)

Token	Devnet Mint Address	Decimals
SOL (native)	So111111111111111111111111111111111111 (wSOL wrapper)	9
USDC	Devnet USDC mint (create via spl-token or use existing devnet USDC)	6
USDT	Devnet USDT mint (create via spl-token or use existing devnet USDT)	6

26.3 Hackathon Submission Checklist

- ☐ Register all team members on colosseum.com by May 4
- ☐ Public GitHub repository with MIT license
- ☐ README.md with: project description, how to run locally, architecture overview, demo link
- ☐ Working devnet deployment
- ☐ Demo video (3-5 minutes) uploaded to YouTube or similar
- ☐ Whitepaper (PDF or link)
- ☐ All content in English
- ☐ Project submission uploaded by May 11, 11:59pm PT

26.4 Reference Documents

Document	Location
Whitepaper	docs/WHITEPAPER.md / docs/WHITEPAPER.pdf
Hackathon Rules	C:\Users\wira0\Downloads\Solana Frontier Hackathon Rules.pdf
Past Hackathon Data	C:\Users\wira0\Downloads\colosseum_all_hackathons.xlsx
Idea Brainstorm	C:\Users\wira0\Downloads\Untitled document (2).pdf